



Maharashtra Education Society's
Institute of Management and Career Courses (IMCC), Pune
AUTONOMOUS



Academic Year 2026-27
FY MCA- SEM I
IT12 Data Structure and Algorithms
Practice Assignment1

Roll No: 2601144

Name: Sanket Jagdish Patil

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def create(self, values):
        for v in values:
            new_node = Node(v)
            if self.head is None:
                self.head = new_node
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next
                temp.next = new_node

    def traverse(self):
        if self.head is None:
            print("List is empty")
            return
        temp = self.head
        while temp:
            print(temp.data, end=" -> " if temp.next else "")
            temp = temp.next
        print()

    def insert_at(self, pos, data):
        if pos < 1:
            print("Invalid position")
            return
        new_node = Node(data)
```



```
if pos == 1:
    new_node.next = self.head
    self.head = new_node
    return
temp = self.head
for _ in range(pos - 2):
    if temp is None:
        break
    temp = temp.next
if temp is None:
    print("Position out of range")
    return
new_node.next = temp.next
temp.next = new_node

def find_middle(self):
    if self.head is None:
        print("List is empty")
        return
    slow = fast = self.head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    print("Middle node value:", slow.data)

def delete(self, value):
    if self.head is None:
        print("List is empty")
        return
    if self.head.data == value:
        self.head = self.head.next
        return
    temp = self.head
    while temp.next and temp.next.data != value:
        temp = temp.next
    if temp.next is None:
        print("Value not found")
        return
    temp.next = temp.next.next

def reverse(self):
    prev = None
    curr = self.head
    while curr:
        nxt = curr.next
        curr.next = prev
```



```
        prev = curr
        curr = nxt
        self.head = prev

    def consecutive_sums(self):
        if self.head is None or self.head.next is None:
            print("Need at least two nodes")
            return
        temp = self.head
        while temp.next:
            print(f"{temp.data} + {temp.next.data} = {temp.data + temp.next.data}")
            temp = temp.next

if __name__ == "__main__":
    ll = LinkedList()

    print("1. Create linked list")
    ll.create([10, 20, 30, 40, 50])

    print("2. Traverse:")
    ll.traverse()

    print("3. Insert 25 at position 3:")
    ll.insert_at(3, 25)
    ll.traverse()

    print("4. Middle node:")
    ll.find_middle()

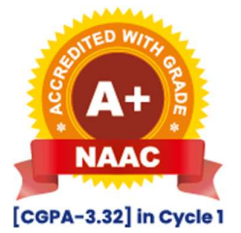
    print("5. Delete node 40:")
    ll.delete(40)
    ll.traverse()

    print("6. Reverse list:")
    ll.reverse()
    ll.traverse()

    print("7. Sum of every two consecutive nodes:")
    ll.consecutive_sums()
```



Maharashtra Education Society's
Institute of Management and Career Courses (IMCC), Pune
AUTONOMOUS



1. Create linked list

2. Traverse:

10 -> 20 -> 30 -> 40 -> 50

3. Insert 25 at position 3:

10 -> 20 -> 25 -> 30 -> 40 -> 50

4. Middle node:

Middle node value: 30

5. Delete node 40:

10 -> 20 -> 25 -> 30 -> 50

6. Reverse list:

50 -> 30 -> 25 -> 20 -> 10

7. Sum of every two consecutive nodes:

50 + 30 = 80

30 + 25 = 55

25 + 20 = 45

20 + 10 = 30